

Q. 3) What are RESTful Web Services? How do they differ from traditional web services? Explain their significance in modern app development. in ASP.NET Core subject.

-> 🖨️ **REST** stands for **Representational State Transfer**.

🖨️ It is an **architecture style** for designing **web services**.

🖨️ RESTful services work using **HTTP protocol** (just like how we browse websites).

🖨️ They use **resources** (like users, products, orders) that are accessed through **URLs (endpoints)**.

🖨️ Each resource can be manipulated using standard **HTTP methods**:

- **GET** → Read data
- **POST** → Create new data
- **PUT/PATCH** → Update data
- **DELETE** → Remove data

Example in ASP.NET Core:

```
[HttpGet("api/products")]
public IEnumerable<Product> GetProducts()
{
    return _context.Products.ToList();
}
```

How do RESTful services differ from Traditional Web Services?

Aspect	Traditional Web Services (SOAP)	RESTful Web Services
Protocol	Uses SOAP (Simple Object Access Protocol)	Uses HTTP directly
Data format	Mostly XML	Can use JSON, XML, or others (JSON is most common)
Complexity	Heavy, strict structure (WSDL, XML schemas)	Simple, lightweight, easy to use
Performance	Slower (because of XML and extra rules)	Faster (because JSON is lightweight)
Ease of Use	More complex to implement	Easy to build and consume
Platform Independence	Supported, but verbose	More flexible and widely adopted

- **Significance of RESTful Web Services in Modern App Development**

1. **Lightweight and Fast** → JSON-based REST APIs are quick, which is important for mobile apps and SPAs (Single Page Applications).
2. **Scalable** → REST APIs are stateless, so apps can easily handle more users by just adding servers.
3. **Cross-platform** → Any app (web, mobile, IoT) can consume a REST API, as long as it understands HTTP.
4. **Easy integration** → Frontend (React, Angular, Vue, even Android/iOS apps) can easily connect with backend REST APIs.
5. **ASP.NET Core** → Provides built-in support for building REST APIs using **Controllers, Routing, Dependency Injection, and Entity Framework Core**.

✓ **In short:**

RESTful Web Services are **modern, simple, and lightweight APIs** built using HTTP methods. They differ from traditional SOAP services by being **faster, easier to implement, and widely used in modern apps**.

Q. 4) How can web service be created in ASP.NET Core? Discuss the steps involved in building and improving the web services.

-> **1. Create a new ASP.NET Core Web API Project**

- Open **Visual Studio** → Create a new project → Select **ASP.NET Core Web API**.
- Configure project name, framework (e.g., .NET 6/7/8).
- Visual Studio generates a template with controllers and Program.cs.

2. Configure Startup & Middleware

- ASP.NET Core uses middleware in the **request pipeline**.
- In Program.cs, we configure services like controllers, Swagger, EF Core, etc.

3. Create a Model (Data Structure)

Example: Product.cs

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

4. Create a Controller (Web API Endpoint)

Controller defines API routes and logic.

5. Run & Test the API

- Press **F5** → API runs on https://localhost:xxxx.
- Use **Swagger UI** (built-in) or **Postman** to test endpoints.
- Example calls:
 - GET /api/products → Returns all products.
 - POST /api/products → Adds a product.
-  **In short:**
 To build a web service in ASP.NET Core → **Create a Web API Project** → **Configure services** → **Create Models** → **Create Controllers** → **Run & Test** → **Improve with DB, DI, Validation, Security, and Documentation.**

Q. 5) Discuss how to support the HTTP patch method in RESTful in web service. what are the best practise for partial updates or resources?

-> Supporting HTTP PATCH in RESTful Web Services

- In REST APIs, PUT is used to **update the whole resource**, while PATCH is used to **partially update** only some fields of a resource.
- ASP.NET Core supports **HTTP PATCH** using JsonPatchDocument<T>.

Best Practices for Partial Updates (PATCH)

1. **Use PATCH only for partial updates**
 - If the client wants to update only **one or two fields**, use PATCH.
 - If the whole object is being updated, use PUT.
2. **Follow JSON Patch standard (RFC 6902)**
 - Use op (operation), path (field), value.
 - Operations: "add", "remove", "replace", "copy", "move", "test".
3. **Validate Input**
 - Ensure patch request is valid and doesn't break resource constraints (e.g., negative price should not be allowed).
4. **Concurrency Handling**
 - Use **ETags or timestamps** to avoid overwriting changes from other clients.
5. **Return Proper HTTP Status Codes**
 - 200 OK → Success with updated resource
 - 400 Bad Request → Invalid patch format

- 404 Not Found → Resource doesn't exist

6. Security

- Limit which properties can be patched (e.g., don't allow patching Id or CreatedDate).
- Validate user permissions before applying changes.

7. Documentation

- Use Swagger/OpenAPI to explain how PATCH requests should be structured.

Q. 2) Explain data caching vs response caching in ASP.NET core ?

-> Data Caching in ASP.NET Core

- **Data caching** means storing frequently used data in memory (RAM) instead of fetching it again and again from the database or API.
- Purpose → To **improve performance** and **reduce database load**.
- Cached data is **temporarily stored** and can be quickly served to the user.

Response Caching in ASP.NET Core

- **Response caching** means caching the **entire HTTP response** (output sent to client) so the same response can be quickly reused for future requests.
- Example: If 100 users request the same product list, the server doesn't process it 100 times — it sends the **cached response**.
- It reduces **server load** and **response time**.

• Difference Between Data Caching & Response Caching

Feature	Data Caching	Response Caching
What is cached?	Data objects (from DB/API)	Entire HTTP response (HTML, JSON, etc.)
Where used?	Inside application (services, repositories)	At response/output level
Purpose	Reduce DB/API calls	Reduce duplicate response generation
Control	Developer controls what to cache	Framework caches whole response
Example	Caching product list from DB	Caching API response <code>/api/products</code>